E2E Service System - Deployment Guide

1. Infrastructure Requirements

Minimum (Pilot)

- Web Server: 2 vCPU, 4GB RAM
- Database: PostgreSQL 15+, 20GB
- Cache: Redis 7+, 2GB RAM
- Storage: 50GB (S3)

Recommended (Production)

- Web Server: 4 vCPU, 8GB RAM (auto-scaling)
- Database: PostgreSQL 15+ (Multi-AZ), 100GB
- Cache: Redis 7+ (cluster), 4GB RAM
- Storage: 500GB (S3 + CDN)

2. Deployment Options

AWS

- Compute: ECS Fargate
- Database: RDS PostgreSQL
- Cache: ElastiCache Redis
- Storage: S3 + CloudFront
- Cost: $300-800/month

DigitalOcean

- Compute: App Platform
- Database: Managed PostgreSQL
- Cache: Managed Redis
- Storage: Spaces + CDN
- Cost: $150-400/month

3. Environment Variables

Backend

```
NODE_ENV=production
PORT=3000
DATABASE_URL=postgresql://...
REDIS_HOST=...
REDIS_PORT=6379
JWT_SECRET=...
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
AWS_S3_BUCKET=e2e-files
SENDGRID_API_KEY=...
STRIPE_SECRET_KEY=...
```

Frontend

```
NEXT_PUBLIC_API_URL=https://api.e2e.com
NEXT_PUBLIC_SENTRY_DSN=...
```

4. Deployment Steps (AWS)

Step 1: Setup Database

```
aws rds create-db-instance \
  --db-instance-identifier e2e-db \
  --engine postgres \
  --master-username admin \
  --master-user-password <password>
```

Step 2: Setup Redis

```
aws elasticache create-cache-cluster \
  --cache-cluster-id e2e-redis \
  --engine redis
```

Step 3: Setup S3

```
aws s3 mb s3://e2e-files
```

Step 4: Deploy Backend

```
docker build -t e2e-backend .
docker push <ecr-url>/e2e-backend:latest
aws ecs update-service --force-new-deployment
```

Step 5: Deploy Frontend

```
npm run build
aws s3 sync out/ s3://e2e-web
aws cloudfront create-invalidation
```

5. CI/CD Pipeline

GitHub Actions

```yaml
name: Deploy

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: npm test
      - run: docker build -t e2e-backend .
      - run: docker push <ecr-url>/e2e-backend
      - run: aws ecs update-service --force-new-deployment
```

6. Database Migrations

```
# Run migrations
npx prisma migrate deploy

# Seed data
npx prisma db seed
```

7. Monitoring

CloudWatch Alarms

- High CPU (> 80%)
- High error rate (> 5%)
- Database connections (> 80%)

Sentry

- Error tracking
- Performance monitoring

8. Backup Strategy

- Daily automated backups (7-day retention)
- S3 versioning enabled
- Point-in-time recovery

9. Scaling

Horizontal Scaling

- Auto-scaling based on CPU/memory
- Load balancer distributes requests

Database Scaling

- Read replicas for analytics

- Partition by tenant_id

10. Security

- SSL/TLS certificate (ACM)
- WAF (Web Application Firewall)
- Security groups (restrict access)
- Secrets Manager (store credentials)

11. Post-Deployment Checklist

- [ ] All services running
- [ ] Database migrations applied
- [ ] SSL certificate installed
- [ ] DNS records updated
- [ ] Monitoring alerts configured
- [ ] Backup strategy enabled
- [ ] Load testing completed
- [ ] Security scan passed